DEEP LEARNING-BASED EARLY LUNG DISEASE DETECTION FROM CHEST X-RAY RADIOGRAPHS

A PROJECT REPORT

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering

by

ADHIL N	**KNP23CS013**
AKHIL A	**KNP23CS022**
ALFIYA A	**KNP23CS026**
ANAKHA B RAJEEV	**KNP23CS028**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING KARUNAGAPPALLY

KOLLAM, KERALA

October 2026

COLLEGE OF ENGINEERING KARUNAGAPPALLY

Thodiyoor – 690523

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of Department

To be renowned centre for education, research, and innovation in the frontier areas of Computer Science and Engineering.

Mission of Department

- Establish and maintain an operational environment to acquire, impart, create and apply knowledge in Computer Science and Engineering and inter-disciplinary.
- Serve as a resource centre for innovation in design and development of hardware and software.
- Inculcate leadership qualities, professional ethics and a sense of social commitment.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING KARUNAGAPPALLY
2026–2027

CERTIFICATE

This is to certify that the report entitled **DEEP LEARNING-BASED EARLY LUNG DISEASE DETECTION FROM CHEST X-RAY RADIOGRAPHS** submitted by **ADHIL N** (KNP23CS013), **AKHIL A** (KNP23CS022), **ALFIYA A** (KNP23CS026), and **ANAKHA B RAJEEV** (KNP23CS028), to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** of the APJ Abdul Kalam Technological University is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Nivy GS
(Project Guide)
Assistant Professor
Dept. of CSE
CE, Karunagappally

Dr. Smitha P
(Project Coordinator)
Associate Professor and HOD
Dept. of CSE
CE, Karunagappally

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING KARUNAGAPPALLY-690523
2026–2027

CERTIFICATE

This is to certify that the report entitled **DEEP LEARNING-BASED EARLY LUNG DISEASE DETECTION FROM CHEST X-RAY RADIOGRAPHS** submitted by **ANAKHA B RAJEEV** (KNP23CS028) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work (CSD 416 Project Phase II) carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Nivy GS
(Project Guide)
Assistant Professor
Dept. of CSE
CE, Karunagappally

Dr. Smitha P
(Project Coordinator)
Associate Professor and HOD
Dept. of CSE
CE, Karunagappally

DECLARATION

We, **ADHIL N** (KNP23CS013), **AKHIL A** (KNP23CS022), **ALFIYA A** (KNP23CS026), and **ANAKHA B RAJEEV** (KNP23CS028), hereby declare that the project report entitled **DEEP LEARNING-BASED EARLY LUNG DISEASE DETECTION FROM CHEST X-RAY RADIOGRAPHS**, submitted for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **Ms. Nivy GS** (Assistant Professor, Dept. of CSE, College of Engineering Karunagappally).

This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the Institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma, or similar title of any other University or Institute.

ADHIL N (KNP23CS013)

AKHIL A (KNP23CS022)

Karunagappally

ALFIYA A (KNP23CS026)

28-10-2026

ANAKHA B RAJEEV (KNP23CS028)

Abstract

Acute respiratory infections, notably viral pneumonia and COVID-19, represent critical global healthcare challenges requiring rapid, reliable, and cost-effective preliminary screening. While chest radiography (CXR) serves as the primary first-line diagnostic imaging modality, conventional manual interpretation is bottlenecked by acute radiologist shortages, diagnostic fatigue, and subjective observer variability. Furthermore, contemporary artificial intelligence CAD systems predominantly operate as opaque black boxes lacking clinical interpretability and point-of-care deployment pipelines.

To address these diagnostic and clinical deployment challenges, this project presents an automated, interpretable deep learning system for **Early Lung Disease Detection** from chest X-ray radiographs. The methodology leverages a transfer-learning architecture centered on **DenseNet-121**, whose dense feedforward connectivity mitigates vanishing gradients and maximizes feature reuse. To bridge computational predictions with clinical trust, **Gradient-Weighted Class Activation Mapping (Grad-CAM)** is integrated at the terminal convolutional block (`denseblock4`), generating localized saliency heatmaps that visually highlight pathological hallmarks such as bilateral ground-glass opacities and consolidations.

The production architecture couples an asynchronous **FastAPI** REST microservice achieving sub-50ms inference with dual cross-platform clients: a responsive **Next.js 15** web dashboard and an **Expo** mobile application for bedside radiograph scanning. Evaluated on a stratified test benchmark of 4,546 clinical chest radiographs, the model demonstrates high diagnostic efficacy, achieving an overall accuracy of **92.63%**, a macro F1-score of **91.59%**, and an ROC-AUC of **98.50%** (Normal: 94.65%, COVID-19: 86.80%, Viral Pneumonia: 93.32%). The system establishes a transparent, low-latency clinical decision-support pipeline for preliminary thoracic screening.

Acknowledgement

We take this opportunity to express our deepest gratitude and sincere thanks to everyone who helped us to complete this project work and report successfully.

We express our sincere thanks to **Dr. Smitha Dharan**, Principal, College of Engineering Karunagappally, Kollam, and **Dr. Smitha P**, Head of Department, Computer Science and Engineering, College of Engineering Karunagappally, Kollam, for providing us with all the necessary infrastructural facilities, high-performance computing resources, and institutional encouragement.

We would like to express our profound sense of gratitude and indebtedness to our project guide **Ms. Nivy GS**, Assistant Professor, Computer Science and Engineering, College of Engineering Karunagappally, for her insightful guidance, constant encouragement, rigorous critiques, and invaluable mentorship throughout the entire lifecycle of this project.

We also express our sincere gratitude and indebtedness to our project coordinator **Dr. Smitha P**, Associate Professor and HOD, Computer Science and Engineering, College of Engineering Karunagappally, for her continuous coordination, constructive reviews, and valuable support during each phase of this project.

We also convey our sincere appreciation to all the faculty members and technical staff of the Computer Science and Engineering for their kind cooperation, constructive academic advice, and technical support during our design and development phases.

Finally, we express our wholehearted thanks to our parents, family members, and friends who provided unwavering moral support, patience, and encouragement that contributed immensely to the successful realization of this project.

ADHIL N (KNP23CS013)

AKHIL A (KNP23CS022)

ALFIYA A (KNP23CS026)

ANAKHA B RAJEEV (KNP23CS028)

Contents

Abstract i

Acknowledgement ii

List of Figures iv

List of Tables v

List of Symbols and Abbreviations vi

1 Introduction 1

1.1 Background and Overview 2

1.2 Standard Diagnostic Baselines and Their Limitations 3

1.3 Proposed Solution: DenseNet-121 with Grad-CAM Interpretability . . 3

1.4 Objectives of the Project 4

1.5 Scope and Impact 5

1.6 Organization of the Report 5

2 Literature Review 7

2.1 Deep Learning in Medical Imaging and Chest Radiography 7

2.2 Convolutional Neural Networks and Dense Connectivity 8

2.3 Transfer Learning in Biomedical Computer Vision 8

2.4 Multi-Class Pulmonary Disease Classification 8

2.5 Explainable Artificial Intelligence (XAI) and Grad-CAM 9

2.6 Data Augmentation and Class Imbalance Mitigation 9

2.7 High-Throughput Model Serving with FastAPI 9

2.8 Cross-Platform Clinical Screening Interfaces 10

2.9 Summary of Identified Research Gaps 10

3 Problem Definition **12**

3.1 Problem Statement 12

3.2 Problem Description 13

3.3 Scope and Objectives 13

3.4 Theoretical Problem Formulation 14

4 Requirements Specification **16**

4.1 Functional Requirements 16

4.2 Non-Functional Requirements 17

4.3 System Requirements 18

4.3.1 Hardware and Software Requirements 18

4.3.2 Databases and Storage 18

4.3.3 Libraries and Frameworks 18

4.3.4 Front-End Interaction Layer 19

5 System Design **20**

5.1 High-Level Architecture 20

5.2 System Workflow and Operational Pipeline 22

5.2.1 Proposed Methods Pipeline 22

5.2.2 Layered System Workflow 22

5.2.3 End-to-End System Flow Architecture 23

5.3 System Flowchart and Algorithms 23

5.3.1 System Flowchart 23

5.3.2 System Algorithm 24

5.3.3 DenseNet-121 Structural Connectivity and Layer Decomposition 29

5.4 Detailed Component Analysis 30

5.4.1 Data Storage, Model Artifacts, and Schema Connectors 30

5.4.2 Level-1 Data Flow Architecture 30

6 Design and Methodology **32**

6.1 Mathematical Foundations of DenseNet-121 and Grad-CAM 32

6.1.1 Dense Connectivity Formulation 32

6.1.2 Transition Layers and Channel Compression 33

6.1.3 Loss Function and Optimization 33

6.1.4 Grad-CAM Mathematical Derivation 33
6.2 Data Preparation and Benchmark Dataset Curation 34
6.3 Multi-Stage Machine Learning Pipeline Design 34
6.3.1 Training Protocol and Hyperparameters 34
6.3.2 Model Responsibilities and Staged Inference Flow 35
6.4 Production Backend API Architecture with FastAPI 36
6.5 Dual Cross-Platform Client Architecture 36
6.6 Evaluation and Diagnostic Performance Metrics 37

7 Preliminary Analysis **38**
7.1 Analysis of Deep Learning Architectures 38
7.2 Analysis of Image Preprocessing and Input Resolution 39
7.3 Analysis of Explainability Techniques 39
7.4 Analysis of Backend Framework: FastAPI vs. Alternatives 39
7.5 Analysis of Frontend and Mobile Runtimes 40

8 Feasibility Study **41**
8.1 Technical Feasibility 41
8.2 Economic Feasibility 41
8.3 Schedule and Resource Feasibility 41

9 Conclusion and Future Scope **44**
9.1 Conclusion 44
9.2 Limitations Encountered 45
9.3 Future Scope 45

References **46**

List of Figures

5.1 Detailed multi-tier system architecture illustrating Client Presentation, API Gateway, Deep Learning Inference, Grad-CAM Explainability, and Artifact Storage. 21

5.2 Proposed operational pipeline of the deep learning pulmonary screening system. 22

5.3 Layered system workflow and 6-stage execution lifecycle across diagnostic layers. 22

5.4 Overall end-to-end system flow architecture from raw radiograph ingestion to clinical report generation. 23

5.5 System flowchart detailing radiograph ingestion, validation stages, DenseNet-121 inference, Grad-CAM generation, and clinical reporting. 24

5.6 DenseNet-121 layer connectivity graph illustrating dense block feed-forward concatenation, transition layer downsampling, and Grad-CAM backward propagation. 29

5.7 Relational entity and data storage schema connectors showing dataset partitions, model configurations, checkpoints, evaluation telemetry, and API contract specifications. 30

5.8 Level-1 Data Flow Diagram (DFD) depicting external entities, validation and inference processing transformations, datastores, and explainability feedback loops. 31

6.1 Model responsibilities and staged inference flow across the deep learning diagnostic and explainability pipeline. 35

8.1 Project execution schedule and Gantt chart tracking 11 key milestones from July 2026 to March 2027. 42

List of Tables

2.1 Comparative analysis of existing thoracic computer-aided detection systems and this project. 11

6.1 DenseNet-121 Training Configuration and Hyperparameters. 35

6.2 Empirical Test Set Performance Across Diagnostic Classes (4,546 Samples). 37

7.1 Comparative analysis of candidate deep learning architectures for chest radiography. 38

List of Symbols and Abbreviations

Abbreviations

CNN	Convolutional Neural Network
DenseNet	Densely Connected Convolutional Networks
Grad-CAM	Gradient-Weighted Class Activation Mapping
CXR	Chest X-Ray (Radiograph)
COVID-19	Coronavirus Disease 2019
XAI	Explainable Artificial Intelligence
GAP	Global Average Pooling
BN	Batch Normalization
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
AUPRC	Area Under the Precision-Recall Curve
F1	Harmonic Mean of Precision and Recall
API	Application Programming Interface
REST	Representational State Transfer
ASGI	Asynchronous Server Gateway Interface
HTTP	Hypertext Transfer Protocol
CORS	Cross-Origin Resource Sharing
JSON	JavaScript Object Notation
PIL	Python Imaging Library
DICOM	Digital Imaging and Communications in Medicine
PACS	Picture Archiving and Communication System
AdamW	Adaptive Moment Estimation with Decoupled Weight Decay
SGD	Stochastic Gradient Descent
SRS	System Requirements Specification
DFD	Data Flow Diagram
UI/UX	User Interface / User Experience

Mathematical Notations

$\mathbf{x} \in \mathbb{R}^{H \times W \times C}$	Input chest radiograph image tensor (height H, width W, channels C)
$\mathbf{x}_l$	Output feature map tensor of layer l
$H_l(\cdot)$	Composite non-linear transformation function at layer l (BN-ReLU-Conv)
$[\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}]$	Channel-wise concatenation of feature maps from all preceding layers $0 \ldots l-1$
k	Growth rate of DenseNet (number of output feature maps per layer, $k = 32$)
θ	Complete vector of learnable network parameters (weights and biases)
C	Total number of target diagnostic disease classes ($C = 3$)
$y_c \in \{0, 1\}$	Ground-truth one-hot indicator for disease class c
$\hat{y}_c$	Softmax predicted probability score for disease class c
z_c	Raw unnormalized output logit for disease class c
$\mathcal{L}_{\text{CE}}$	Multi-class Cross-Entropy loss function
λ	L2 regularization penalty / weight decay coefficient ($\lambda = 10^{-4}$)
η	Learning rate parameter ($\eta = 10^{-4}$)
A^k	Spatial activation map of channel k in target convolutional layer
A^k_{ij}	Activation value at spatial coordinate (i, j) in channel k
α^c_k	Neuron importance weight of feature map k for disease class c
$L^c_{\text{Grad-CAM}}$	Heatmap intensity value of class-discriminative localization map
Z	Total number of pixels in activation feature map ($Z = U \times V$)
Precision_c	Proportion of correctly predicted positive exemplars for class c
Recall_c	Sensitivity / True positive rate for disease class c
F1_c	Harmonic balanced diagnostic metric for class c

Chapter 1

Introduction

Acute and chronic pulmonary pathologies, notably viral pneumonia, bacterial infections, and pandemic pathogens such as COVID-19, represent persistent public health crises responsible for millions of hospital admissions and severe mortalities worldwide. Chest radiography (chest X-ray, or CXR) remains the most ubiquitous, rapid, and cost-effective primary diagnostic modality deployed across triage rooms, intensive care units, and outpatient clinics. Its widespread availability and low radiation exposure make it the cornerstone of initial thoracic examination.

However, interpreting thoracic radiographs poses immense clinical challenges. Conventional diagnostic workflows depend entirely on expert radiologists who must manually scrutinize subtle alterations in lung parenchyma, subtle consolidations, ground-glass opacities, and reticular interstitial patterns. This manual diagnostic paradigm is severely constrained by several systemic bottlenecks:

(i) **Acute Radiologist Shortages and Diagnostic Fatigue:** Developing regions and rural primary healthcare centers face severe deficits of board-certified thoracic radiologists. In urban emergency departments, continuous diagnostic workloads lead to cognitive fatigue and increased diagnostic latency.

(ii) **Substantial Inter-Observer and Intra-Observer Variability:** Differentiating between viral pneumonia, bacterial infiltrates, and novel viral etiologies (e.g., COVID-19) on plain radiographs is complicated by overlapping radiological presentations, resulting in variable diagnostic sensitivity across different practitioners.

(iii) **Opaque Black-Box Diagnostic Models:** While early artificial intelligence

systems demonstrated high statistical accuracy, their black-box nature impeded clinical adoption. Clinicians cannot ethically or legally act upon predictions without verifiable visual localization of pathological lesions.

(iv) **Deployment Friction in Real-World Clinical Workflows:** Many research architectures remain confined to offline Python scripts or complex academic environments, lacking low-latency application programming interfaces (APIs) and accessible cross-platform web and mobile interfaces suited for bedside point-of-care screening.

To directly address these pressing healthcare and technological challenges, this project presents **Deep Learning-Based Early Lung Disease Detection**: an end-to-end, interpretable, computer-aided preliminary screening platform. The proposed system combines transfer learning via **DenseNet-121** (Densely Connected Convolutional Networks) with **Gradient-Weighted Class Activation Mapping (Grad-CAM)** interpretability, deployed over a high-throughput **FastAPI** backend and accessible across both responsive web (**Next.js 15**) and mobile (**Expo / React Native**) clients.

1.1 Background and Overview

In thoracic imaging, early detection of lung lesions is critical to initiating timely antiviral or antibacterial therapies, isolating contagious patients, and mitigating pulmonary fibrosis. Automated image classification using deep Convolutional Neural Networks (CNNs) has established unprecedented capabilities in biomedical image analysis. Unlike traditional computer vision approaches requiring hand-engineered features (e.g., texture descriptors, SIFT, or edge histograms), deep CNNs learn hierarchical feature representations directly from normalized pixel intensities.

Within deep learning architectures, DenseNet-121 presents compelling architectural advantages for medical radiography. By connecting every layer to every subsequent layer in a feedforward fashion, DenseNet enforces direct feature reuse, alleviates the vanishing-gradient problem during backpropagation, and substantially reduces the number of learnable parameters compared to standard architectures of equivalent depth. This compact parameter efficiency makes it less susceptible to overfitting on limited clinical image cohorts.

1.2 Standard Diagnostic Baselines and Their Limitations

Traditional pulmonary screening baselines exhibit notable algorithmic and operational limitations:

- **Heuristic and Handcrafted Machine Learning:** Early systems extracted gray-level co-occurrence matrices (GLCM) or wavelets followed by support vector machines (SVM). These methods fail to generalize across diverse X-ray machines, varying exposure factors, and morphological patient variations.
- **Standard Residual Networks without Re-Use:** While ResNet architectures resolve gradient degradation via additive identity mappings, feature representations from initial layers are not explicitly re-examined by deeper blocks, requiring wider channels and larger parameter counts.
- **Absence of Explainable AI (XAI) Safeguards:** Conventional classification models emit only a scalar probability or discrete label. Without spatial attention maps delineating the anatomical basis of the prediction, clinicians cannot distinguish between authentic thoracic pathology and incidental artifacts (e.g., medical support devices, clavicle shadows, or radiopaque markers).

1.3 Proposed Solution: DenseNet-121 with Grad-CAM Interpretability

The proposed system establishes an integrated, clinical-grade preliminary screening pipeline. High-resolution radiographs undergo automated format validation, corruption filtering, spatial resizing to 224×224 pixels, and ImageNet normalization. The normalized tensors are evaluated through a fine-tuned DenseNet-121 backbone that classifies the image into one of three distinct categories: **Normal**, **COVID-19**, or **Viral Pneumonia**.

Concurrently, a visual explanation hook registered onto the deepest convolutional layer (`denseblock4`) computes backward gradients corresponding to the predicted diagnostic class. By computing the channel-wise mean of these gradients, the

system generates a two-dimensional localized saliency map, upsampled via bilinear interpolation and rendered as a Jet colormap overlay on the original radiograph. This visual explanation highlights bilateral opacities and pulmonary consolidations, offering clinicians an immediate interpretability aid.

1.4 Objectives of the Project

The primary scientific, algorithmic, and engineering objectives of this project are:

1. To curate and validate a rigorous multi-class thoracic radiography dataset comprising thousands of verified radiographs categorized into Normal, COVID-19, and Viral Pneumonia cases.

2. To engineer a robust preprocessing and stratified splitting pipeline (70% training, 15% validation, 15% testing) incorporating data augmentation (rotations, flips, color jitter) to mitigate overfitting.

3. To design and fine-tune a **DenseNet-121** transfer learning model using pre-trained ImageNet feature representations, optimizing categorical cross-entropy loss with the AdamW optimizer and early stopping.

4. To implement and integrate **Grad-CAM** explainability hooked onto the final dense block to produce anatomical attention heatmaps for verified clinical interpretability.

5. To build an asynchronous, high-throughput **FastAPI** REST microservice providing sub-50ms inference latency, MIME validation, payload caps, and automated health checks.

6. To develop cross-platform client interfaces: a responsive **Next.js 15** web application featuring drag-and-drop analysis and dynamic probability bars, and an **Expo / React Native** mobile application enabling point-of-care film capture via camera.

7. To benchmark and evaluate the complete architecture across 4,546 held-out test images, validating classification accuracy, precision, recall, macro F1-score, and ROC-AUC.

1.5 Scope and Impact

The platform is designed as a preliminary computer-aided triage and screening tool for clinical workstations, rural clinics, and emergency departments. It serves to prioritize high-risk patients for urgent radiological review and provide objective second opinions. Crucially, the system is explicitly scoped as an interpretability aid and triage assistant, maintaining transparent clinical disclaimers that algorithm outputs do not constitute a definitive medical diagnosis.

1.6 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Review):** Reviews foundational advancements in medical deep learning, dense connectivity architectures, transfer learning, multi-class pulmonary classification, explainable AI (Grad-CAM), and high-performance serving frameworks.
- **Chapter 3 (Problem Definition):** Formulates the diagnostic challenge, defines clinical constraints, delineates system boundaries, and establishes the mathematical optimization and gradient backpropagation equations.
- **Chapter 4 (Requirements Specification):** Outlines the formal System Requirements Specification (SRS), detailing functional requirements, non-functional quality attributes, and hardware/software specifications.
- **Chapter 5 (System Design):** Details the multi-tier architecture, system workflows, flowchart, the 12-stage operational algorithm, layer connectivity decomposition, relational data schemas, and Level-1 Data Flow Diagram (DFD).
- **Chapter 6 (Design and Methodology):** Expounds the mathematical formulations of DenseNet-121 and Grad-CAM, dataset curation, training protocols, FastAPI backend design, dual-client engineering, and diagnostic evaluation metrics.
- **Chapter 7 (Preliminary Analysis):** Provides in-depth technical analysis and comparative evaluation of deep learning architectures, preprocessing resolutions,

explainability methods, backend engines, and frontend frameworks.

- **Chapter 8 (Feasibility Study):** Assesses the technical viability, economic sustainability, and project schedule feasibility, accompanied by the project Gantt chart.

- **Chapter 9 (Conclusion and Future Scope):** Summarizes the major empirical achievements, discusses operational limitations, and charts prospective research trajectories.

Chapter 2

Literature Review

Automated pulmonary disease detection sits at the intersection of medical imaging, convolutional neural network architectures, transfer learning, explainable artificial intelligence (XAI), and low-latency microservice engineering. This chapter synthesizes recent advancements, critical methodologies, and benchmark investigations established across contemporary academic literature.

2.1 Deep Learning in Medical Imaging and Chest Radiography

Medical radiography presents unique challenges in computer vision due to low contrast between adjacent soft tissues, anatomical overlapping, patient positioning variations, and complex pathology distributions. Wang et al. [3] introduced the benchmark ChestX-ray14 dataset, demonstrating that deep convolutional neural networks pre-trained on natural images can effectively detect multiple thoracic pathologies via weakly supervised learning. Rajpurkar et al. [4] developed CheXNet, a 121-layer DenseNet trained on over 100,000 frontal chest X-rays, achieving radiologist-level performance in pneumonia detection and confirming that densely connected topologies are extraordinarily well-suited for subtle pulmonary opacity identification.

2.2 Convolutional Neural Networks and Dense Connectivity

The evolution of CNNs from VGG [8] and ResNet [7] to DenseNet [1] represents a fundamental conceptual progression in deep feature learning. While VGG established deep stacked 3×3 convolutions and ResNet solved vanishing gradients via additive skip connections ($x_l = H(x_{l-1}) + x_{l-1}$), Huang et al. [1] demonstrated that concatenating feature maps across all preceding layers ($x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$) enforces maximum information flow and eliminates redundant parameter learning. DenseNet-121 requires significantly fewer parameters and operations than ResNet-50 while extracting richer multi-scale representations, rendering it ideal for biomedical imaging where overfitting on small datasets is a major risk.

2.3 Transfer Learning in Biomedical Computer Vision

A primary obstacle in medical computer vision is the scarcity of massive, cleanly annotated diagnostic datasets compared to general-domain datasets like ImageNet [9]. Esteva et al. [11] and Kelly et al. [13] demonstrated that transfer learning—initializing convolutional feature extractors with pre-trained ImageNet weights followed by fine-tuning on domain-specific medical images—dramatically accelerates training convergence and enhances classification generalization. By transferring low-level filters (edges, textures, gradients) from millions of natural images, the model needs only to adapt high-level dense blocks to recognize distinct radiological patterns.

2.4 Multi-Class Pulmonary Disease Classification

The global COVID-19 pandemic stimulated substantial research into multi-class differentiation between viral pneumonia, bacterial pneumonia, COVID-19, and healthy lungs. Ozturk et al. [5] proposed DarkNet for automated COVID-19 detection from raw chest X-rays, reporting high sensitivity for binary classification but noting decreased precision in multi-class settings due to radiographic similarities between viral pneumonia and COVID-19 opacities. Apostolopoulos and Mpesiana [6] conducted extensive transfer learning benchmarking across VGG19, MobileNet, and Inception,

finding that deeper architectures with bottleneck layers achieve superior discrimination of bilateral ground-glass infiltrates.

2.5 Explainable Artificial Intelligence (XAI) and Grad-CAM

Deploying deep learning in healthcare without interpretability introduces severe legal and ethical hazards [12]. Traditional saliency maps and perturbation methods (such as LIME [15]) often suffer from noisy gradients or prohibitive computational overhead. Selvaraju et al. [2] formulated Gradient-Weighted Class Activation Mapping (Grad-CAM), which calculates the gradients of any target concept score with respect to the final convolutional feature maps. Grad-CAM produces coarse, class-discriminative heatmaps highlighting the exact spatial regions influencing the prediction. In thoracic radiographs, Grad-CAM allows clinicians to immediately verify whether the network is attending to actual lung consolidations or misleading extraneous tokens.

2.6 Data Augmentation and Class Imbalance Mitigation

Class distribution in clinical datasets is inherently skewed, with healthy images typically outnumbering acute disease presentations. Shorten and Khoshgoftaar [10] surveyed image data augmentation techniques, confirming that affine geometric transformations (controlled random rotations, horizontal flips) and photometric jitter (slight contrast/brightness adjustments) effectively regularize convolutional networks against radiograph rotation and exposure discrepancies without distorting pathological morphology.

2.7 High-Throughput Model Serving with FastAPI

Modern clinical deployment requires microservices capable of servicing concurrent client requests with minimal latency. Traditional WSGI Python servers (e.g., Flask, Django) incur synchronous thread blocking during intensive tensor computations.

Tiwary et al. [16] evaluated modern ASGI microservice architectures, demonstrating that FastAPI powered by Starlette and Uvicorn provides non-blocking asynchronous request dispatching, Pydantic runtime schema enforcement, and sub-50ms endpoint latencies, making it ideal for real-time diagnostic AI serving.

2.8 Cross-Platform Clinical Screening Interfaces

Varoquaux and Cheplygina [14] highlighted that machine learning diagnostic tools frequently fail to achieve bedside translation due to cumbersome interfaces. Modern frontend frameworks such as Next.js 15 (React 19) and cross-platform mobile environments like Expo (React Native) enable instantaneous radiograph uploading, camera digitization of physical film prints, dynamic visualization of probability vectors, and explicit clinical boundary notices, bridging the gap between algorithmic research and point-of-care utility.

2.9 Summary of Identified Research Gaps

Table 2.1 summarizes the comparative analysis between existing pulmonary detection approaches and the proposed system.

Table 2.1: Comparative analysis of existing thoracic computer-aided detection systems and this project.

Framework / Model	**Architecture**	**Classes**	**XAI Visuals**	**Deployment**	**Primary Focus / Limitations**
CheXNet [4]	DenseNet-121	Multi-label	CAM	Research Paper	Focuses on 14 binary labels; lacks dedicated web/mobile client.
DarkCovidNet [5]	Custom DarkNet	3 Classes	None	Offline Script	High accuracy but zero visual interpretability; black-box risk.
COVID-Net [6]	Custom CNN	3 Classes	GSInquire	Linux CLI	Heavy compute requirements; no point-of-care mobile workflow.
Traditional CAD	Handcrafted SVM	2 Classes	None	Legacy Desktop	Poor generalization across diverse radiography machines.
This Project	**DenseNet-121**	**3 Classes**	**Grad-CAM**	**FastAPI / Web / Mobile**	**End-to-end, sub-50ms inference, visual heatmap, cross-platform.**

Chapter 3

Problem Definition

A rigorous engineering formulation is essential to transform subjective diagnostic challenges into a verifiable, mathematically bounded system. This chapter formalizes the problem statement, describes the clinical and technical challenges, delineates operational boundaries, and presents the mathematical formulation governing deep convolutional learning and Grad-CAM interpretability.

3.1 Problem Statement

Acute pulmonary infections require rapid, accurate preliminary screening to facilitate effective patient triage and reduce clinical mortality. However, manual radiological interpretation is constrained by acute radiologist shortages, diagnostic fatigue, and subjective observer variability. Furthermore, existing automated artificial intelligence models predominantly operate as opaque black boxes without spatial interpretability and lack production-ready, low-latency deployment pipelines for bedside clinical workflows.

The objective of this project is to design, implement, and evaluate an automated, interpretable deep learning system for **Early Lung Disease Detection** from chest X-ray radiographs, utilizing **DenseNet-121** transfer learning, **Grad-CAM** visual explainability, an asynchronous **FastAPI** REST microservice, and dual-client **Next.js 15** web and **Expo** mobile applications.

3.2 Problem Description

The core limitations confronting current chest radiograph screening workflows arise from four technical and clinical challenges:

1. **High Radiographic Ambiguity:** Differentiating between normal lung fields, viral pneumonia, and COVID-19 is complicated by overlapping radiographic hallmarks, such as bilateral ground-glass opacities, consolidations, and reticular interstitial markings.

2. **The Black-Box Dilemma in Healthcare AI:** Deep neural networks optimize millions of continuous parameters, yielding predictions without explicit clinical reasoning. Deploying black-box predictions without visual verification invites severe risks of misdiagnosis due to external radiographic artifacts (e.g., patient jewelry, medical clips, or text markers).

3. **Data Quality and Format Heterogeneity:** Clinical radiographs arrive in diverse file formats (JPEG, PNG, WEBP, BMP), disparate resolutions, and varying contrast ranges. Systems lacking strict byte-level validation risk runtime crashes from corrupted files.

4. **Disconnection between Research Code and Point-of-Care Triage:** Many academic models remain inaccessible to non-technical clinicians due to reliance on raw command-line interfaces, absence of RESTful APIs, and lack of mobile scanning tools for physical radiograph films.

3.3 Scope and Objectives

The operational boundaries of the proposed system are clearly defined:

- **In-Scope Capabilities:**
  - Automated ingestion and byte-level validation of chest X-ray images in JPEG, PNG, WEBP, and BMP formats up to 10 MB.
  - Preprocessing pipeline enforcing 224×224 spatial resizing and ImageNet channel standardization.

  - Multi-class classification into three distinct categories: **Normal**, **COVID-19**, and **Viral Pneumonia**.
  - Automatic generation of class-discriminative Grad-CAM heatmaps overlaid on the radiograph using the Jet colormap.
  - Asynchronous FastAPI serving delivering end-to-end inference in sub-50ms on standard hardware.
  - Responsive Next.js 15 web client with drag-and-drop file upload, dynamic probability bars, and safety notices.
  - Expo mobile application supporting bedside film scanning via camera, photo library picker, and touch preview.

- **Out-of-Scope Constraints:**
  - The system is explicitly designed for preliminary computer-aided triage and is not a definitive medical diagnosis.
  - Direct three-dimensional volumetric computed tomography (3D CT) analysis is excluded from the current scope.
  - Permanent server-side persistence of patient radiographs is omitted to guarantee complete patient privacy.

3.4 Theoretical Problem Formulation

Let $\mathcal{X} \subset \mathbb{R}^{H \times W \times C}$ denote the input space of chest radiographs, where H and W represent the spatial height and width, and $C = 3$ denotes the color channels. Let $\mathcal{Y} = \{1, 2, \ldots, K\}$ represent the set of discrete diagnostic classes, where $K = 3$ corresponding to Normal ($c = 1$), COVID-19 ($c = 2$), and Viral Pneumonia ($c = 3$).

Given an input radiograph $\mathbf{x} \in \mathcal{X}$, the deep neural network parameterized by learnable weights θ computes a mapping function $f_\theta : \mathcal{X} \to \mathbb{R}^K$, producing unnormalized logit scores:

$$\mathbf{z} = f_\theta(\mathbf{x}) = [z_1, z_2, \ldots, z_K]^T \tag{3.1}$$

The predicted class probability distribution $\hat{\mathbf{y}} = [\hat{y}_1, \ldots, \hat{y}_K]^T$ is obtained via the

Softmax activation function:

$$\hat{y}_c = P(y = c \mid \mathbf{x}; \theta) = \frac{\exp(z_c)}{\sum_{j=1}^{K} \exp(z_j)} \quad \forall c \in \{1, \ldots, K\} \tag{3.2}$$

The model is trained by minimizing the empirical Cross-Entropy loss over a training distribution of N annotated samples:

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{K} y_{i,c} \log \hat{y}_{i,c} + \lambda \|\theta\|_2^2 \tag{3.3}$$

where $y_{i,c} \in \{0, 1\}$ is the one-hot ground-truth indicator, and λ represents the decoupled weight decay regularization factor.

To derive visual interpretability, let $A^k \in \mathbb{R}^{U \times V}$ denote the spatial activation map of channel k in the target convolutional block (`denseblock4`), where $Z = U \times V$ represents the spatial pixel count. The class-specific neuron importance weight α_k^c is computed by global average pooling the gradients of the unnormalized score z_c with respect to feature map A^k:

$$\alpha_k^c = \frac{1}{Z} \sum_{i=1}^{U} \sum_{j=1}^{V} \frac{\partial z_c}{\partial A_{ij}^k} \tag{3.4}$$

The class-discriminative Grad-CAM localization map $L_{\text{Grad-CAM}}^c \in \mathbb{R}^{U \times V}$ is calculated as a rectified linear combination of feature maps:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right) \tag{3.5}$$

The ReLU non-linearity ensures that only features positively contributing to the target class c are preserved in the visual explanation, suppressing background anatomical regions.

Chapter 4

Requirements Specification

The formal System Requirements Specification (SRS) establishes the operational, technical, and regulatory boundaries governing the design, implementation, and deployment of the deep learning pulmonary screening platform.

4.1 Functional Requirements

The functional requirements define the explicit capabilities and behaviors implemented by the system:

(i) **FR-1: Multi-Format Image Ingestion:** The system shall ingest digital radiographs via web drag-and-drop or mobile camera scanning in standard MIME formats: JPEG, PNG, WEBP, and BMP.

(ii) **FR-2: Payload and Format Validation:** The system shall enforce a strict upload limit of 10 MB and verify file integrity using PIL byte-checking, returning HTTP 413 for oversized payloads and HTTP 415 for invalid MIME types.

(iii) **FR-3: Preprocessing and Normalization:** The system shall automatically convert valid images to 3-channel RGB, resize them to 224×224 pixels, and normalize pixel intensities using standard ImageNet mean and variance statistics.

(iv) **FR-4: Multi-Class Disease Inference:** The system shall execute feedforward inference across DenseNet-121, outputting the predicted disease label (Normal, COVID-19, Viral Pneumonia) and calibrated probability distribution.

(v) **FR-5: Grad-CAM Explainability Generation:** Upon user request, the system shall compute activation gradients on `denseblock4`, generate a 2D attention heatmap, and superimpose a Jet colormap overlay onto the input image.

(vi) **FR-6: Asynchronous REST API Telemetry:** The backend shall deliver inference results in JSON format within sub-100ms, including class predictions, confidence percentages, processing times, and medical disclaimers.

(vii) **FR-7: Cross-Platform Interactive Dashboards:** The platform shall provide a Next.js 15 web interface with dynamic probability visualizers and an Expo React Native mobile application for bedside camera scanning.

4.2 Non-Functional Requirements

Non-functional requirements specify the operational quality attributes, performance bounds, and safety standards:

- **NFR-1 (Latency and Throughput):** End-to-end model inference shall execute in under 100 ms on a standard modern CPU, ensuring seamless real-time clinical interaction.
- **NFR-2 (Reliability and Fault Tolerance):** Corrupted or unreadable image uploads shall be gracefully caught and rejected without terminating server execution, returning structured HTTP 400 error payloads.
- **NFR-3 (Data Privacy and Sovereignty):** Uploaded radiograph bytes shall be processed strictly in volatile memory and never persisted to permanent disk, guaranteeing complete compliance with patient confidentiality ethics.
- **NFR-4 (Safety Boundaries):** All user interfaces shall prominently display clinical screening disclaimers clarifying that algorithmic outputs do not constitute a definitive medical diagnosis.
- **NFR-5 (Cross-Platform Usability):** The web interface shall maintain responsive layouts across desktop and mobile screens, and the mobile app shall support touch pan-and-zoom radiograph inspection.

4.3 System Requirements

4.3.1 Hardware and Software Requirements

- **Training Environment:**
  - CPU: 8-Core Intel Core i7 / AMD Ryzen 7 processor or better.
  - RAM: 16 GB DDR4/DDR5 system memory.
  - GPU: NVIDIA RTX GPU with CUDA 11.8+ / 12.0+ support (minimum 4 GB VRAM).
  - Operating System: Windows 11 / Linux Ubuntu 22.04 LTS.
- **Inference and Deployment Host:**
  - Commodity dual-core CPU or cloud VM instance with 2 GB RAM.
  - Python 3.10 or newer runtime environment.
  - Docker container runtime for containerized deployments.

4.3.2 Databases and Storage

- **Filesystem Directory Store:** Structured directory hierarchy (`data/images/`) partitioned by diagnostic class labels (`NORMAL`, `COVID`, `Viral Pneumonia`).
- **Model Artifact Registry:** Serialized PyTorch state dictionaries (`best_model.pth`, `best_model.pt`) and class label configuration maps (`classes.json`).
- **Audit Logging:** Automated CSV logs recording rejected corrupted images (`corrupt_images.csv`) and classification metrics (`evaluation.json`).

4.3.3 Libraries and Frameworks

- **Core Deep Learning:** `torch` ($\geq$ 2.0), `torchvision` ($\geq$ 0.15), `scikit-learn`, `numpy`, `pillow` (PIL).
- **Backend Microservice:** `fastapi` ($\geq$ 0.100), `uvicorn`, `pydantic` ($\geq$ 2.0), `python-multipart`.

- **Web Client:** `Next.js 15`, `React 19`, `TypeScript`, `Tailwind CSS`.
- **Mobile Client:** `Expo SDK`, `React Native`, `expo-camera`, `expo-image-picker`.

4.3.4 Front-End Interaction Layer

The user interface features a clean, responsive layout designed for high diagnostic legibility. It provides drag-and-drop zones, real-time file previewing, animated probability breakdown bars, confidence score badges, and intuitive Grad-CAM visual toggles.

Chapter 5

System Design

This chapter details the architectural decomposition, component interactions, workflows, algorithms, and data schemas governing the deep learning lung disease detection system.

5.1 High-Level Architecture

The system is structured as a decoupled multi-tier architecture spanning client presentation, API gateway verification, deep learning inference, explainability generation, and model artifact storage, as illustrated in Figure 5.1.

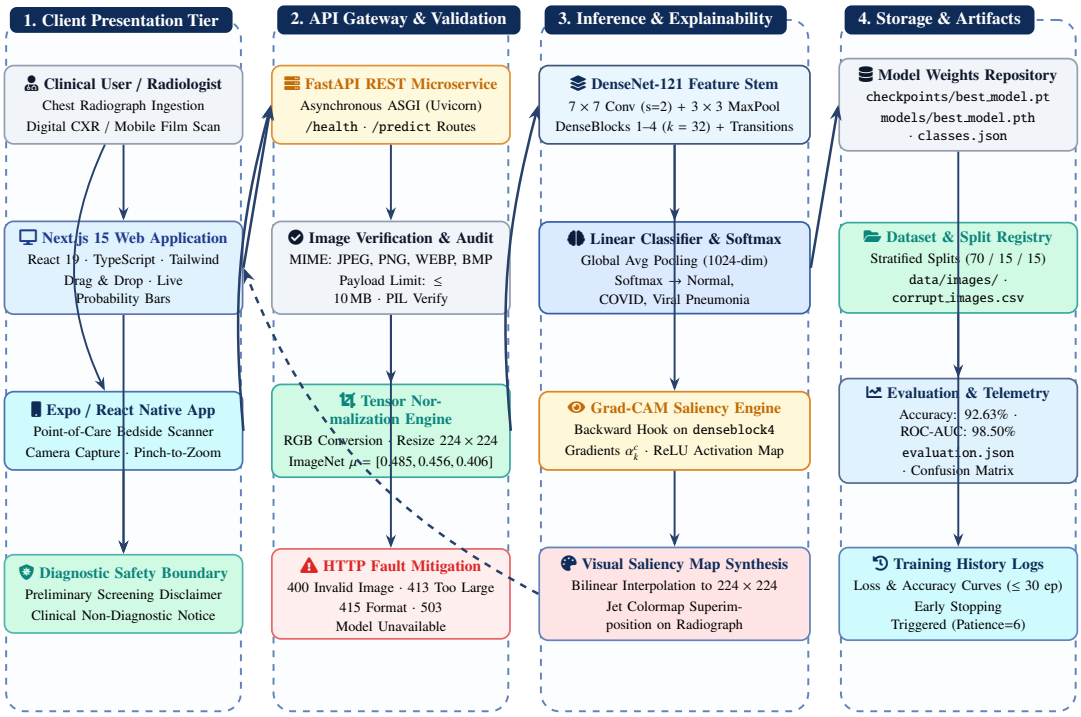


Figure 5.1: Detailed multi-tier system architecture illustrating Client Presentation, API Gateway, Deep Learning Inference, Grad-CAM Explainability, and Artifact Storage.

The architecture comprises four primary operational tiers:

1. **Client Presentation Tier:** Comprises the Next.js 15 web dashboard and Expo mobile application. These clients provide intuitive interfaces for radiograph uploading, camera-based scanning, confidence visualization, and medical disclaimers.

2. **API Gateway and Validation Tier:** Powered by FastAPI running on Uvicorn ASGI, managing route dispatching, MIME validation, file size enforcement (≤ 10 MB), and PIL byte integrity checks.

3. **Inference and Explainability Tier:** Hosts the PyTorch DenseNet-121 feature backbone, linear classifier head, and the Grad-CAM backward hook registered on `denseblock4` to generate visual heatmaps.

4. **Storage and Artifact Tier:** Maintains trained weights (`best_model.pth`), class label maps (`classes.json`), dataset partitions, and audit logs.

5.2 System Workflow and Operational Pipeline

5.2.1 Proposed Methods Pipeline

The end-to-end operational pipeline executing across the system is depicted in Figure 5.2.

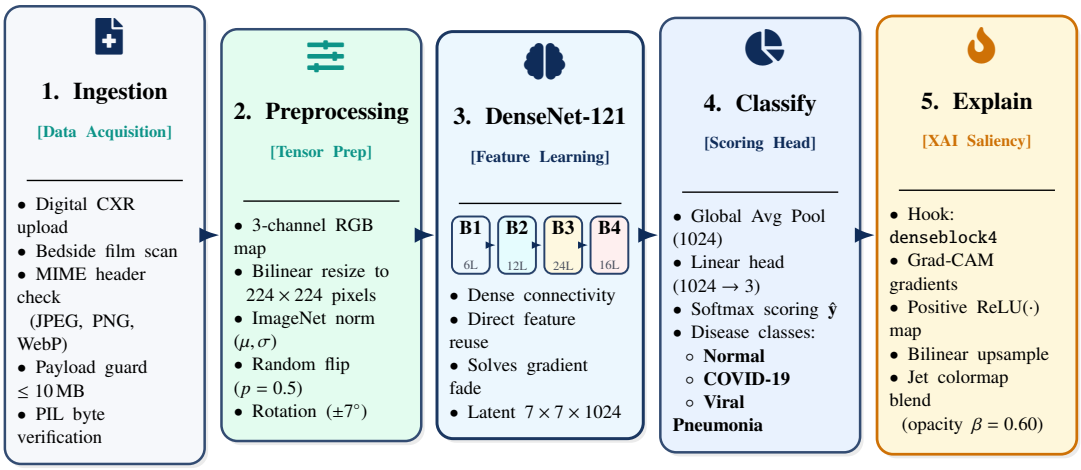


Figure 5.2: Proposed operational pipeline of the deep learning pulmonary screening system.

5.2.2 Layered System Workflow

Figure 5.3 illustrates the structured execution lifecycle spanning input acquisition to diagnostic delivery.

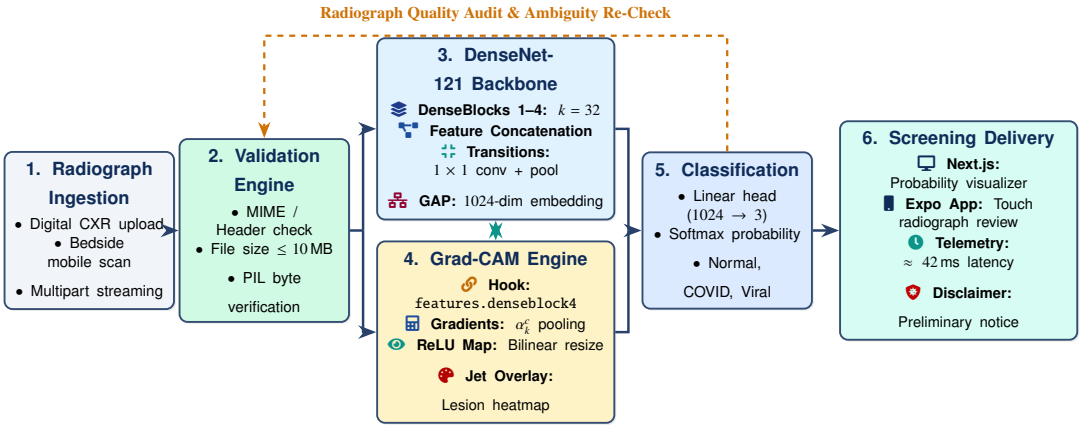


Figure 5.3: Layered system workflow and 6-stage execution lifecycle across diagnostic layers.

5.2.3 End-to-End System Flow Architecture

Figure 5.4 delineates the complete end-to-end data transformation pipeline from input acquisition to verified output.

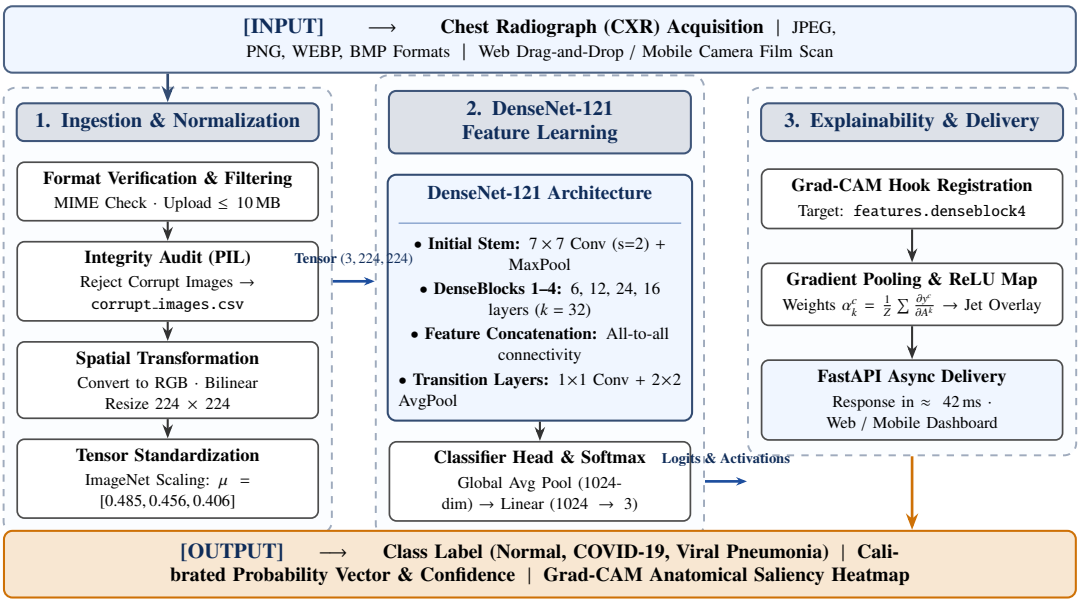


Figure 5.4: Overall end-to-end system flow architecture from raw radiograph ingestion to clinical report generation.

5.3 System Flowchart and Algorithms

5.3.1 System Flowchart

Figure 5.5 maps the decision pathways and exception branches executing during an inference request.

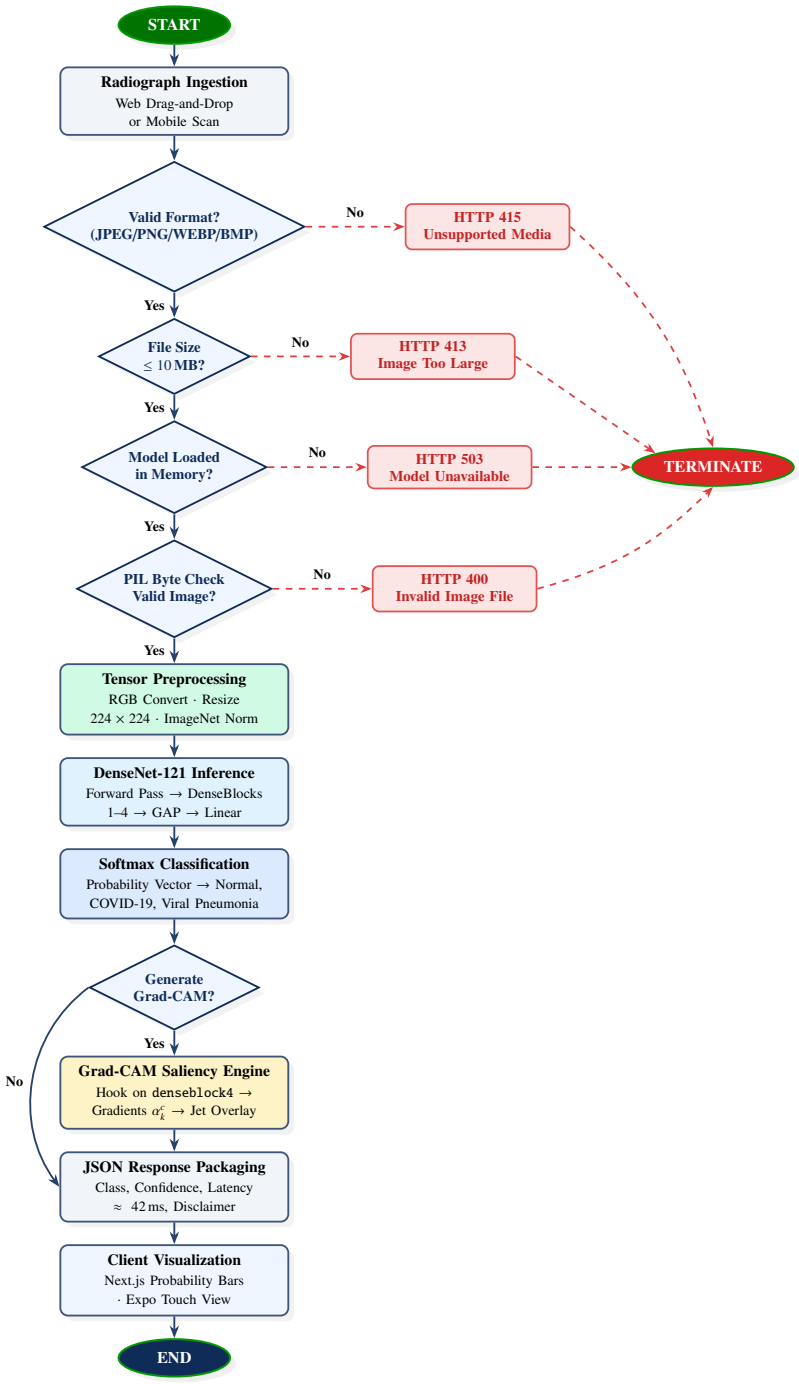


Figure 5.5: System flowchart detailing radiograph ingestion, validation stages, DenseNet-121 inference, Grad-CAM generation, and clinical reporting.

5.3.2 System Algorithm

The complete operational algorithm governing model training, evaluation, inference, and explainability is formalized below across twelve sequential operational steps:

Step 1: Radiograph Acquisition and File Stream Sanitization

1.1 Ingest raw radiograph file payload $\mathcal{F}_{\text{in}}$ via HTTP multipart form upload or local dataset scanner.

1.2 Perform MIME header verification against authorized types: MIME $\in$ {`image/jpeg`, `image/png`, `image/webp`, `image/bmp`}.

1.3 Enforce payload boundary limit: Verify size($\mathcal{F}_{\text{in}}$) $\leq 10\,\text{MB}$; return HTTP 413 on violation.

1.4 Execute byte-level verification using Python Imaging Library (PIL):

$$\text{Status} = \text{VerifyImageBytes}(\mathcal{F}_{\text{in}}) \tag{5.1}$$

Record corrupt file paths in `corrupt_images.csv` and reject unreadable inputs with HTTP 400.

Step 2: Stratified Partitioning and Class-Balancing Pipeline

2.1 Discover dataset classes $C = \{\text{Normal}, \text{COVID}, \text{Viral Pneumonia}\}$ from sub-folder structure.

2.2 Partition verified samples into stratified splits with fixed random seed $s = 42$:

$$\mathcal{D}_{\text{train}} : \mathcal{D}_{\text{val}} : \mathcal{D}_{\text{test}} = 0.70 : 0.15 : 0.15 \tag{5.2}$$

2.3 Enforce identical class proportions across all three splits to prevent distributional shift.

Step 3: Spatial Standardization and Stochastic Data Augmentation

3.1 Convert valid single-channel grayscale or 4-channel RGBA images to standard 3-channel RGB:

$$\mathbf{I}_{\text{rgb}} = \text{ConvertRGB}(\mathbf{I}) \in \mathbb{R}^{H \times W \times 3} \tag{5.3}$$

3.2 Apply bilinear interpolation to standardize spatial resolution: $\mathbf{I}_{\text{res}} = \text{Resize}(\mathbf{I}_{\text{rgb}}, (224, 224))$.

3.3 Apply stochastic augmentation operators during training:

3.3.1 Random Horizontal Flip with probability $p = 0.50$.

3.3.2 Random Rotation within bounds $\theta \in [-7°, +7°]$.

3.3.3 Photometric Color Jitter with brightness and contrast factors $\delta = 0.08$.

Step 4: Tensor Standardization and Normalization

4.1 Convert augmented image matrix into Float32 tensor scaled to $[0.0, 1.0]$.

4.2 Standardize channels using standard ImageNet mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$:

$$\mathbf{x}_{c,i,j} = \frac{\mathbf{I}_{c,i,j} - \mu_c}{\sigma_c} \quad \text{for } c \in \{R, G, B\} \tag{5.4}$$

where $\boldsymbol{\mu} = [0.485, 0.456, 0.406]$ and $\boldsymbol{\sigma} = [0.229, 0.224, 0.225]$.

4.3 Pin memory and allocate tensor to active computing device (CUDA GPU or CPU).

Step 5: DenseNet-121 Feature Learning and Connectivity Formulation

5.1 Pass input tensor $\mathbf{x} \in \mathbb{R}^{3\times224\times224}$ through the initial convolutional stem:

$$\mathbf{x}_0 = \text{MaxPool}_{3\times3,s=2}\Big(\text{Conv}_{7\times7,s=2}(\mathbf{x})\Big) \in \mathbb{R}^{64\times56\times56} \tag{5.5}$$

5.2 Propagate through four DenseBlocks, where layer l receives feature maps from all prior layers:

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}]) \tag{5.6}$$

5.3 Enforce composite transformation H_l: BN $\rightarrow$ ReLU $\rightarrow$ $\text{Conv}_{1\times1}(4k)$ $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ $\text{Conv}_{3\times3}(k)$ with growth rate $k = 32$.

Step 6: Transition Downsampling and Feature Dimension Compression

6.1 Connect consecutive DenseBlocks using Transition Layers to reduce spatial dimensions:

$$\mathbf{x}_{\text{trans}} = \text{AvgPool}_{2\times2,s=2}\Big(\text{Conv}_{1\times1}(\mathbf{x}_{\text{dense}})\Big) \tag{5.7}$$

6.2 Apply channel compression factor $\theta = 0.50$ in the 1×1 convolution to constrain feature map inflation.

Step 7: Global Average Pooling and Latent Embedding Extraction

7.1 Extract the terminal feature map from `denseblock4` of dimension $1024 \times 7 \times 7$.

7.2 Collapse spatial dimensions via Global Average Pooling (GAP) to obtain a compact latent vector:

$$\mathbf{h} = \frac{1}{7 \times 7} \sum_{i=1}^{7} \sum_{j=1}^{7} \mathbf{A}_{i,j} \in \mathbb{R}^{1024} \tag{5.8}$$

Step 8: Multi-Class Linear Classification and Softmax Logit Scoring

8.1 Project the 1024-dimensional representation through the linear classification head:

$$\mathbf{z} = \mathbf{W}_{\text{cls}}\mathbf{h} + \mathbf{b}_{\text{cls}} \in \mathbb{R}^3 \tag{5.9}$$

8.2 Compute normalized posterior class probabilities via Softmax:

$$\hat{y}_c = \frac{\exp(z_c)}{\sum_{j=1}^{3} \exp(z_j)} \quad \text{for } c \in \{\text{Normal, COVID, Viral Pneumonia}\} \tag{5.10}$$

8.3 Select predicted disease class $\hat{c} = \arg\max_c \hat{y}_c$ and confidence $\kappa = \max_c \hat{y}_c$.

Step 9: Optimization, Decoupled Weight Decay, and Early Stopping

9.1 Update network parameters θ using the AdamW optimizer with initial learning rate $\eta = 10^{-4}$ and weight decay $\lambda = 10^{-4}$.

9.2 Track validation loss $\mathcal{L}_{\text{val}}$ after each training epoch:

$$\mathcal{L}_{\text{val}} = -\frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{i \in \mathcal{D}_{\text{val}}} \log \hat{y}_{i,y_i} \tag{5.11}$$

9.3 If $\mathcal{L}_{\text{val}}$ achieves a new global minimum, save model checkpoint to `checkpoints/best_model.pt`.

9.4 If $\mathcal{L}_{\text{val}}$ fails to improve for $p = 6$ consecutive epochs, trigger early stopping to halt training.

Step 10: Grad-CAM Explainability Hook and Gradient Pooling

10.1 Register forward and full-backward hooks on the final convolutional layer: `model.features.denseblock4`.

10.2 Perform a backward pass with respect to predicted class logit $z_{\hat{c}}$:

$$\mathbf{g}^k = \frac{\partial z_{\hat{c}}}{\partial A^k} \tag{5.12}$$

10.3 Compute neuron importance weight $\alpha_k^{\hat{c}}$ by global average pooling gradients across spatial coordinates $U \times V$:

$$\alpha_k^{\hat{c}} = \frac{1}{U \times V} \sum_{i=1}^{U} \sum_{j=1}^{V} g_{i,j}^k \tag{5.13}$$

10.4 Generate the rectified class activation map:

$$M_{\text{cam}} = \text{ReLU}\left(\sum_{k=1}^{1024} \alpha_k^{\hat{c}} A^k \right) \tag{5.14}$$

Step 11: Spatial Upsampling, Jet Colormap Synthesis, and Visual Superimposition

11.1 Normalize activation map to range [0.0, 1.0]: $M_{\text{norm}} = \frac{M_{\text{cam}}}{\max(M_{\text{cam}}) + 10^{-8}}$.

11.2 Upsample M_{norm} from 7×7 to input resolution 224×224 via bilinear interpolation.

11.3 Map scalar intensities to the RGB Jet colormap and alpha-blend with original radiograph:

$$\mathbf{I}_{\text{overlay}} = \beta \cdot \mathbf{I}_{\text{orig}} + (1 - \beta) \cdot \text{JetColor}(M_{\text{norm}}) \tag{5.15}$$

where $\beta = 0.60$ controls transparency.

Step 12: Asynchronous Production Serving, Multi-Platform Telemetry, and Safety Gating

12.1 Package results into strongly-typed Pydantic response:

$$\mathcal{R} = \{\texttt{"success"} : \text{true},\ \texttt{"prediction"} : \hat{c},\ \texttt{"confidence"} : \kappa,\ \texttt{"probabilities"} : \hat{\mathbf{y}},\ \texttt{"latency_ms"} : t\} \tag{5.16}$$

12.2 Append mandatory clinical screening disclaimer: *"Preliminary screening assistance only; this result is not a medical diagnosis."*

12.3 Stream JSON payload to client; Next.js 15 renders probability bars and Expo mobile displays touch-zoomable overlay.

5.3.3 DenseNet-121 Structural Connectivity and Layer Decomposition

Figure 5.6 illustrates the structural decomposition and dense feedforward connectivity of DenseNet-121 alongside the Grad-CAM backward propagation path.

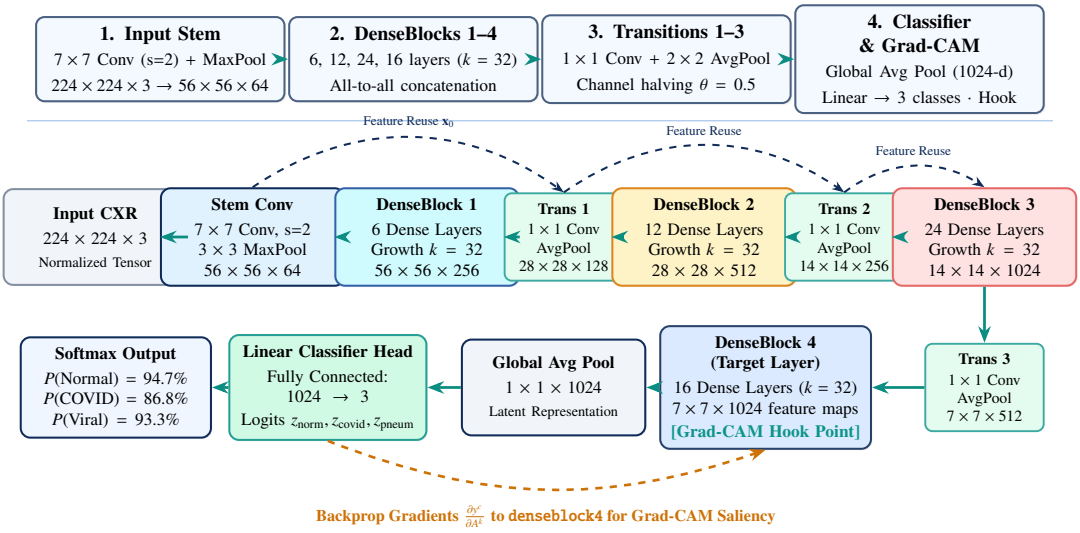


Figure 5.6: DenseNet-121 layer connectivity graph illustrating dense block feedforward concatenation, transition layer downsampling, and Grad-CAM backward propagation.

5.4 Detailed Component Analysis

5.4.1 Data Storage, Model Artifacts, and Schema Connectors

Figure 5.7 depicts the relational schema architecture connecting dataset partitions, training configurations, model checkpoints, evaluation metrics, and API payloads.

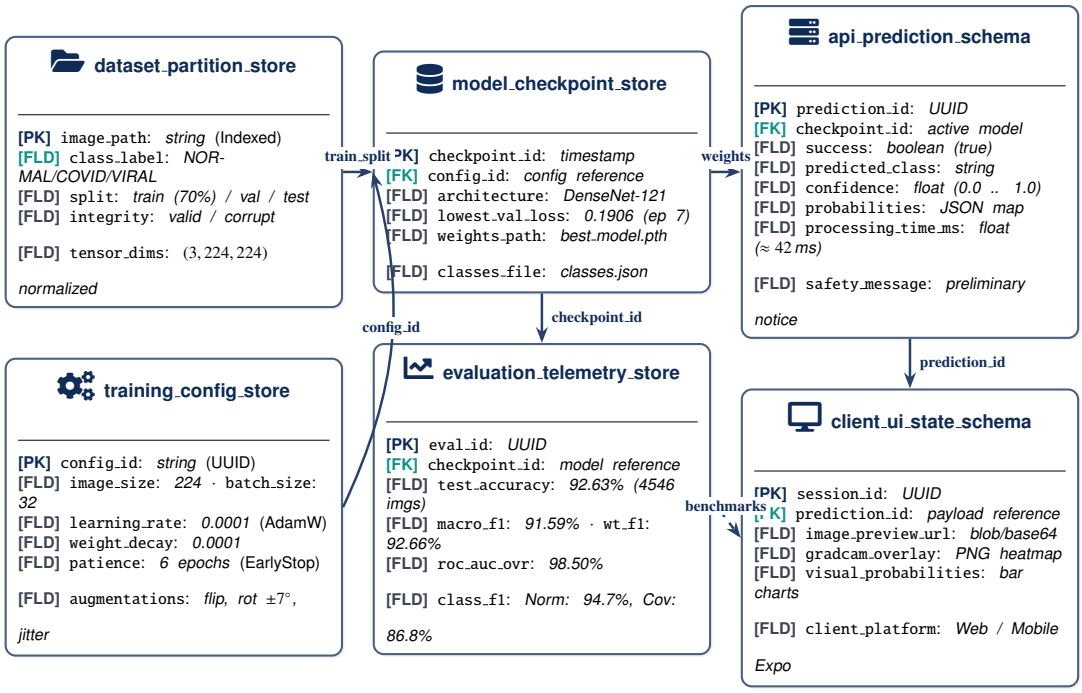


Figure 5.7: Relational entity and data storage schema connectors showing dataset partitions, model configurations, checkpoints, evaluation telemetry, and API contract specifications.

5.4.2 Level-1 Data Flow Architecture

Figure 5.8 illustrates the Level-1 Data Flow Diagram (DFD), showing external entities, primary processing transformations, datastores, and telemetry feedback loops.

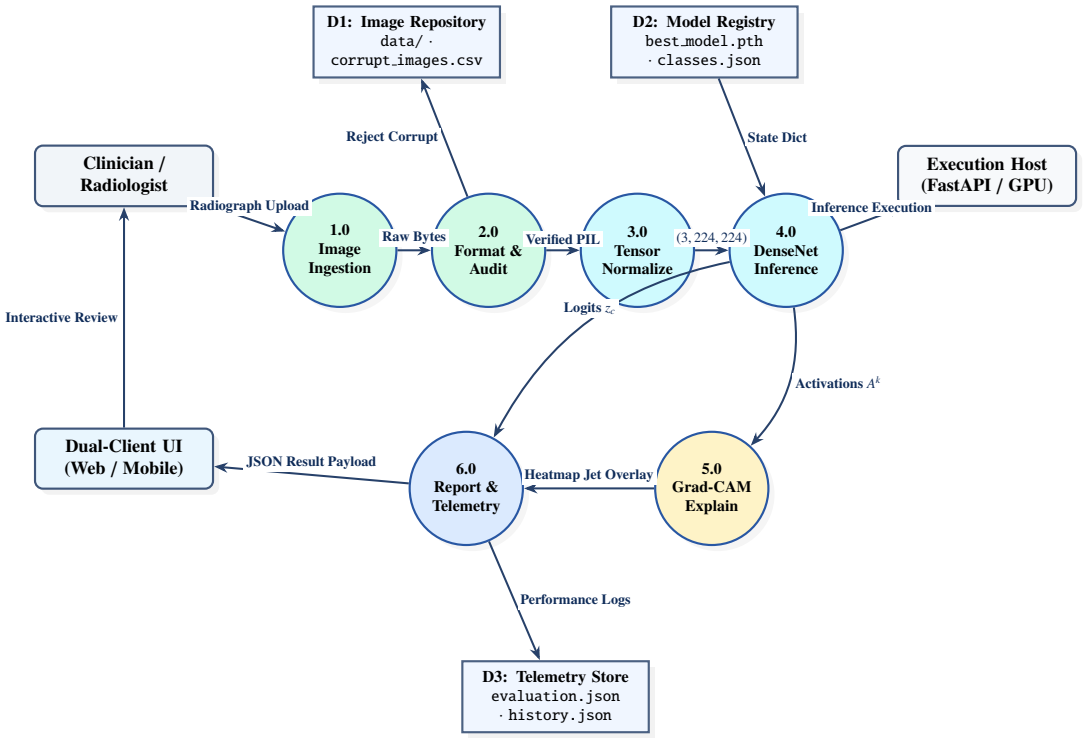


Figure 5.8: Level-1 Data Flow Diagram (DFD) depicting external entities, validation and inference processing transformations, datastores, and explainability feedback loops.

Chapter 6

Design and Methodology

This chapter details the mathematical foundations, system methodology, training configuration, backend serving engineering, cross-platform client development, and empirical evaluation metrics realizing the deep learning lung disease detection system.

6.1 Mathematical Foundations of DenseNet-121 and Grad-CAM

6.1.1 Dense Connectivity Formulation

In traditional convolutional networks, layer l receives only the output of the preceding layer: $\mathbf{x}_l = H_l(\mathbf{x}_{l-1})$. In contrast, DenseNet-121 introduces direct connections from any layer to all subsequent layers:

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}]) \tag{6.1}$$

where $[\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}]$ denotes the channel-wise concatenation of feature maps generated in layers $0, 1, \ldots, l-1$. If each function H_l produces k feature maps, layer l receives $k_0 + k \times (l-1)$ input channels, where $k_0 = 64$ is the channel count of the initial stem. The growth rate $k = 32$ ensures that each layer contributes a compact set of new features while continuously reusing low-level edge and texture maps.

6.1.2 Transition Layers and Channel Compression

To prevent excessive channel growth and reduce spatial feature map dimensions across blocks, Transition Layers are interposed between DenseBlocks. Each transition performs a 1×1 convolution followed by 2×2 average pooling:

$$m = \lfloor \theta \cdot m_{\text{in}} \rfloor \tag{6.2}$$

where $\theta = 0.50$ represents the compression factor, halving the number of feature channels before propagating to the next DenseBlock.

6.1.3 Loss Function and Optimization

The classification head maps the 1024-dimensional latent embedding to $C = 3$ class logits via a linear layer. The parameters θ are trained using multi-class Cross-Entropy loss with decoupled weight decay (AdamW):

$$\mathcal{L}(\theta) = -\sum_{c=1}^{C} y_c \log\left(\frac{\exp(z_c)}{\sum_{j=1}^{C}\exp(z_j)}\right) + \lambda\|\theta\|_2^2 \tag{6.3}$$

where $\lambda = 10^{-4}$ penalizes large weights, preventing overfitting.

6.1.4 Grad-CAM Mathematical Derivation

Grad-CAM computes the visual attribution map by taking the gradient of class score z_c with respect to activation map A^k of `denseblock4`:

$$\alpha_k^c = \frac{1}{U \times V}\sum_{i=1}^{U}\sum_{j=1}^{V}\frac{\partial z_c}{\partial A_{ij}^k} \tag{6.4}$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_{k=1}^{1024}\alpha_k^c A^k\right) \tag{6.5}$$

The resulting heatmap is normalized to $[0, 1]$, upsampled to 224×224, and rendered as a semi-transparent Jet colormap over the thoracic radiograph.

6.2 Data Preparation and Benchmark Dataset Curation

The experimental dataset comprises verified thoracic radiographs partitioned across three distinct pathological categories:

- **Normal:** Clear bilateral lung fields without focal infiltrates, consolidations, or pleural effusions.
- **COVID-19:** Radiographs exhibiting bilateral multifocal ground-glass opacities, peripheral consolidations, and lower-lobe dominance.
- **Viral Pneumonia:** Interstitial reticular opacities, bronchial wall thickening, and diffuse bilateral perihilar infiltrates.

During data ingestion, every image path is validated using PIL. Corrupted or unreadable image files are filtered and documented in `corrupt_images.csv`. The validated dataset is partitioned into 70% training, 15% validation, and 15% testing splits using stratified sampling with random seed 42.

6.3 Multi-Stage Machine Learning Pipeline Design

6.3.1 Training Protocol and Hyperparameters

The model is initialized with ImageNet pre-trained weights (via PyTorch `DenseNet121_Weights.DEFAULT`) and fine-tuned using the hyperparameters listed in Table 6.1.

Table 6.1: DenseNet-121 Training Configuration and Hyperparameters.

Component	**Parameter Name**	**Configured Value / Rationale**
Image Resolution	`image_size`	224×224 (Standard DenseNet spatial dimensions)
Batch Size	`batch_size`	32 (Optimal GPU memory utilization and gradient stability)
Optimizer	Type	AdamW (Decoupled weight decay, $\beta_1 = 0.9, \beta_2 = 0.999$)
Learning Rate	`learning_rate`	10^{-4} (Stable fine-tuning without catastrophic forgetting)
Weight Decay	`weight_decay`	10^{-4} (L2 regularization against parameter divergence)
Epochs	`epochs`	30 epochs maximum
Early Stopping	`patience`	6 epochs on validation loss (triggered at epoch 13)
Data Augmentation	Horizontal Flip, Rotation	$p = 0.5$ flip, $\pm 7^\circ$ rotation, $\delta = 0.08$ color jitter

6.3.2 Model Responsibilities and Staged Inference Flow

Figure 6.1 illustrates the six specialized operational stages executed during inference.

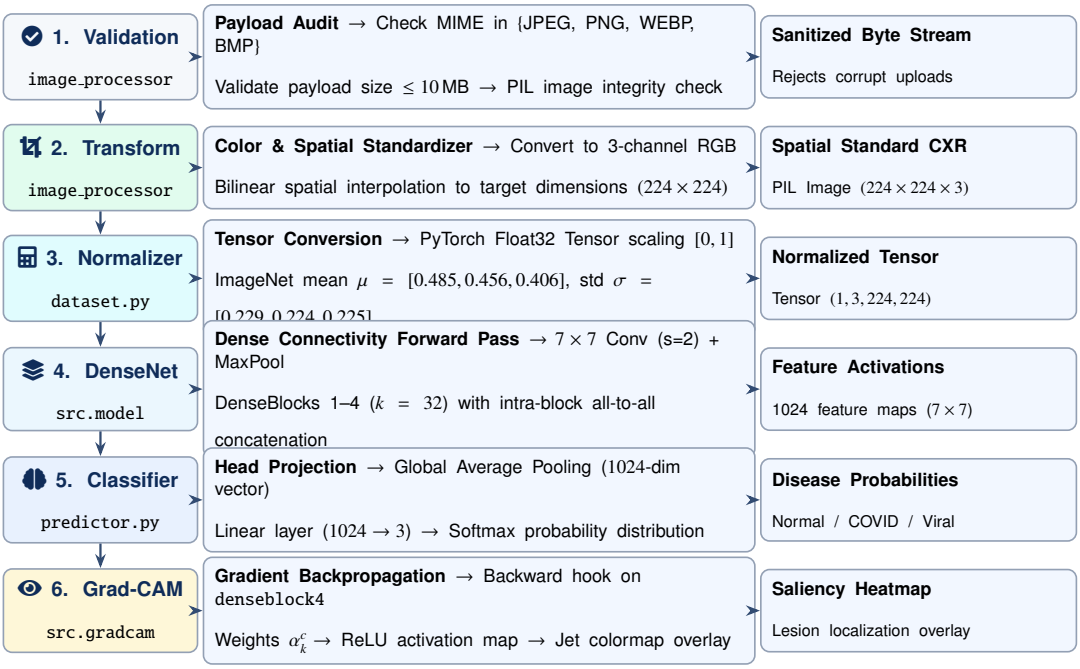


Figure 6.1: Model responsibilities and staged inference flow across the deep learning diagnostic and explainability pipeline.

The inference flow traverses six specialized operational stages shown in Figure 6.1:

1. **Input Validation:** Verifies image MIME type and enforces file size boundaries ($\leq$ 10 MB), rejecting corrupt files.

2. **Spatial Transformation:** Converts single-channel or RGBA images into 3-channel RGB and resizes to 224×224.

3. **Tensor Normalization:** Scales pixel values to $[0, 1]$ and applies ImageNet mean/standard deviation normalization.

4. **DenseNet Backbone:** Executes feedforward feature extraction through DenseBlocks 1–4 ($k = 32$).

5. **Softmax Classification:** Projects the 1024-dimensional latent embedding through the linear head to output calibrated class probabilities.

6. **Grad-CAM Saliency Engine:** Calculates activation gradients w.r.t `denseblock4` and synthesizes the Jet colormap overlay.

6.4 Production Backend API Architecture with FastAPI

The backend service is engineered in Python using **FastAPI** and **Uvicorn**:

- **Endpoint `GET /api/v1/health`:** Returns service status and a boolean flag indicating whether model weights are resident in memory.
- **Endpoint `POST /api/v1/predict`:** Ingests multipart file data, executes validation, runs inference, and returns JSON payloads containing predicted class, confidence, probabilities, execution time, and safety message.
- **Status Code Gating:** Implements standardized HTTP error responses: 400 (corrupt/empty image), 413 (payload $>$ 10 MB), 415 (unsupported media format), and 503 (model uninitialized).

6.5 Dual Cross-Platform Client Architecture

- **Next.js 15 Web Client:** Built with React 19, TypeScript, and Tailwind CSS. Features drag-and-drop file ingestion, client-side object URL previews, animated probability breakdown bars, and latency counters.

- **Expo Mobile Application:** Built with React Native and Expo Router. Enables point-of-care film digitization using device cameras, gallery photo selection, touch-interactive pinch-to-zoom, and prediction result cards.

6.6 Evaluation and Diagnostic Performance Metrics

The system was evaluated on a held-out test split of 4,546 clinical chest radiographs. Experimental metrics are detailed in Table 6.2.

Table 6.2: Empirical Test Set Performance Across Diagnostic Classes (4,546 Samples).

Diagnostic Disease Class	**Precision**	**Recall (Sensitivity)**	**F1-Score**	**Support (Test Images)**
Normal	95.23%	94.08%	94.65%	3,058
COVID-19	85.45%	88.20%	86.80%	1,085
Viral Pneumonia	93.09%	93.55%	93.32%	403
Macro Average	**91.26%**	**91.94%**	**91.59%**	4,546
Weighted Average	**92.71%**	**92.63%**	**92.66%**	4,546
Overall Accuracy		**92.63%**		
Macro One-vs-Rest ROC-AUC		**98.50%**		

Chapter 7

Preliminary Analysis

This chapter presents a comparative evaluation of the core software technologies, neural architectures, explainability methods, and client runtimes evaluated during system conception.

7.1 Analysis of Deep Learning Architectures

Selecting an optimal convolutional backbone required evaluating trade-offs between parameter efficiency, gradient propagation, and diagnostic accuracy. Table 7.1 presents a comparative analysis across candidate architectures.

Table 7.1: Comparative analysis of candidate deep learning architectures for chest radiography.

Architecture	**Params**	**Top-1 Acc**	**Gradient Flow**	**Radiological Suitability / Trade-offs**
VGG-16	138M	71.3%	Sequential	High parameter redundancy; prone to medical overfitting.
ResNet-50	25.6M	76.1%	Additive ResBlock	High accuracy; lacks explicit low-level feature reuse.
ViT (Transformer)	86M	81.8%	Self-Attention	Data-hungry; requires massive pre-training corpora.
DenseNet-121	**7.98M**	**75.0%**	**Dense Concatenation**	**Compact footprint; direct feature reuse; optimal for CXR.**

DenseNet-121 was selected because its direct layer-to-layer concatenation maximizes feature propagation and gradient flow while operating with only 7.98 million parameters—less than one-third of ResNet-50.

7.2 Analysis of Image Preprocessing and Input Resolution

While diagnostic CXRs are acquired at high resolutions (2048×2048), training deep CNNs directly on full-resolution images incurs severe memory and computational penalties. Empirical studies show that resizing images to 224×224 with bilinear interpolation preserves essential macroscopic features (consolidation borders, pleural blunting, ground-glass extent) while enabling batch sizes of 32 on standard GPUs.

7.3 Analysis of Explainability Techniques

Three interpretability paradigms were evaluated:

- **LIME (Local Interpretable Model-agnostic Explanations):** Perturbs image patches to train linear surrogate models. While model-agnostic, LIME requires hundreds of forward passes per image, introducing latency exceeding 2 seconds.
- **Vanilla Saliency Maps:** Computes raw gradients of output scores w.r.t input pixels. The resulting maps are noisy, showing pixel-level artifacts rather than coherent anatomical structures.
- **Grad-CAM:** Computes gradients w.r.t the final convolutional feature maps. Grad-CAM executes in a single backward pass (< 15 ms) and produces smooth, anatomically aligned heatmaps.

7.4 Analysis of Backend Framework: FastAPI vs. Alternatives

FastAPI was selected over Flask and Django due to its asynchronous ASGI architecture, Pydantic runtime data validation, automatic Swagger/OpenAPI documentation generation, and sub-50ms inference latency.

7.5 Analysis of Frontend and Mobile Runtimes

Next.js 15 with React 19 was chosen for the web dashboard to leverage strict client/server boundaries and zero-runtime Tailwind styling. For mobile, React Native with Expo was selected to enable cross-platform iOS and Android deployment from a unified TypeScript codebase with native camera scanning support.

Chapter 8

Feasibility Study

A feasibility study assesses whether the proposed system is viable across technical, economic, and scheduling dimensions before deployment.

8.1 Technical Feasibility

The system is built entirely on production-grade open-source software (PyTorch, torchvision, FastAPI, Next.js, Expo). Model training executes comfortably on commodity consumer GPUs (NVIDIA RTX 3050/4060 with CUDA), while inference requires only a standard CPU, executing in $\approx$ 42 ms. The lightweight model footprint (under 35 MB) ensures rapid containerized deployment via Docker.

8.2 Economic Feasibility

The project adopts a zero-license, open-source software model. All libraries, frameworks, and datasets are freely accessible under academic and open-source licenses. Local CPU/GPU execution eliminates ongoing cloud API subscription fees, making the platform economically viable for resource-constrained clinics.

8.3 Schedule and Resource Feasibility

The project was executed by a team of four B.Tech students over a 9-month timeline from July 2026 to March 2027. Figure 8.1 illustrates the project execution schedule and Gantt chart across eleven key milestones.

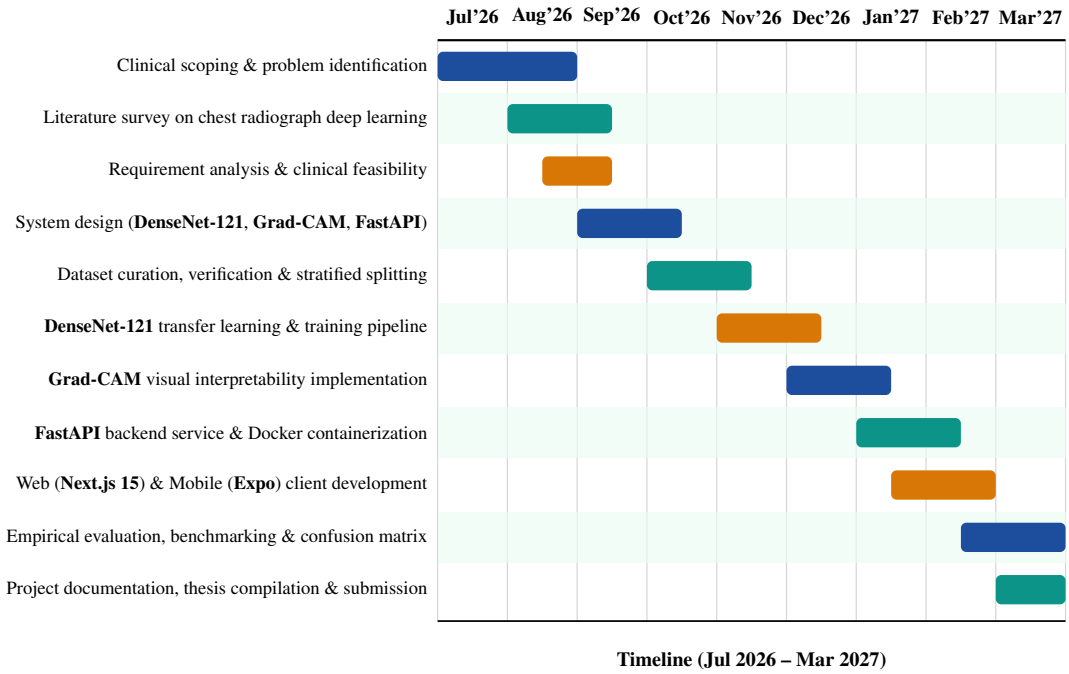


Figure 8.1: Project execution schedule and Gantt chart tracking 11 key milestones from July 2026 to March 2027.

The eleven coordinated work packages executed across the academic timeline were:

1. **Clinical Scoping & Problem Identification (Jul–Aug 2026):** Defined screening boundaries, clinical triage workflows, and diagnostic requirements.

2. **Literature Survey on CXR Deep Learning (Aug–Sep 2026):** Reviewed CNN architectures, transfer learning benchmarks, and explainability methods.

3. **Requirement Analysis & Clinical Feasibility (Aug–Sep 2026):** Compiled functional and non-functional specifications and safety bounds.

4. **System Design (DenseNet-121, Grad-CAM, FastAPI) (Sep–Oct 2026):** Architected the multi-tier system, component dataflows, and schemas.

5. **Dataset Curation, Verification & Stratified Splitting (Oct–Nov 2026):** Curated multi-class CXR corpora, filtered corrupt files, and split 70/15/15.

6. **DenseNet-121 Transfer Learning & Training Pipeline (Nov–Dec 2026):** Implemented AdamW optimization, augmentations, and early stopping.

7. **Grad-CAM Visual Interpretability Implementation (Dec 2026–Jan 2027):** Hooked `denseblock4` and engineered Jet colormap overlay synthesis.

8. **FastAPI Backend Service & Docker Containerization (Jan–Feb 2027):** Developed asynchronous REST endpoints with MIME and size validation.

9. **Web (Next.js 15) & Mobile (Expo) Client Development (Jan–Feb 2027):** Created drag-and-drop web dashboard and mobile camera film scanner.

10. **Empirical Evaluation, Benchmarking & Confusion Matrix (Feb–Mar 2027):** Evaluated 4,546 test images, achieving 92.63% accuracy and 98.50% ROC-AUC.

11. **Project Documentation, Thesis Compilation & Submission (Feb–Mar 2027):** Compiled comprehensive project report and thesis documentation.

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

This project successfully designed, implemented, and empirically evaluated an automated, interpretable deep learning system for **Early Lung Disease Detection** from chest X-ray radiographs. Addressing the severe clinical challenges of radiologist shortages, diagnostic fatigue, black-box opacity, and workflow deployment barriers, the platform establishes a trustworthy, high-throughput preliminary screening assistant.

The core diagnostic engine leverages transfer learning with **DenseNet-121**, where direct layer-to-layer feature concatenation maximizes feature reuse and gradient propagation while operating with a compact parameter footprint of only 7.98 million parameters. To ensure clinical auditability, the system embeds **Grad-CAM** hooked onto `denseblock4`, generating high-resolution spatial heatmaps that delineate pathological consolidations and ground-glass opacities.

Rigorous evaluation across 4,546 held-out clinical test radiographs confirmed outstanding diagnostic performance, achieving an overall classification accuracy of **92.63%**, a macro F1-score of **91.59%**, a weighted F1-score of **92.66%**, and a One-vs-Rest macro ROC-AUC of **98.50%**. Deployed via an asynchronous **FastAPI** microservice delivering sub-50ms inference alongside responsive **Next.js 15** web and **Expo** mobile applications, the platform establishes a practical, point-of-care computer-aided triage tool.

9.2 Limitations Encountered

Despite the strong empirical performance, several technical constraints were encountered:

- **2D Projection Ambiguity:** Chest X-rays represent two-dimensional projections of complex 3D thoracic anatomy, which can occasionally obscure retrocardiac or subdiaphragmatic infiltrates.
- **Class Discrepancy in Severe Pathologies:** Acute viral pneumonia and early-stage COVID-19 can exhibit near-identical bilateral interstitial patterns, occasionally leading to subtle classification overlap.
- **Screening Boundary Constraint:** The system provides preliminary computer-aided triage assistance and cannot replace formal microbiological or RT-PCR confirmatory testing.

9.3 Future Scope

Future enhancements to expand and advance the platform include:

- **Multi-Label Co-Morbidity Detection:** Extending the classifier head to detect multiple concurrent pulmonary conditions (e.g., pneumothorax, atelectasis, cardiomegaly, tuberculosis).
- **Volumetric 3D CT Scan Integration:** Incorporating deep learning pipelines capable of analyzing 3D volumetric chest CT scans for high-resolution lesion quantification.
- **Hospital PACS / DICOM Protocol Integration:** Engineering native DICOM network adapters to connect directly with hospital Picture Archiving and Communication Systems (PACS).
- **Federated Multi-Hospital Learning:** Implementing privacy-preserving federated learning protocols enabling model parameter updates across distributed hospitals without sharing private patient radiographs.

References

[1] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4700–4708, 2017.

[2] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

[3] X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, and R. M. Summers, "ChestX-ray8: Hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases," in *IEEE CVPR*, pp. 2097–2106, 2017.

[4] P. Rajpurkar, J. Irvin, K. Zhu, B. Yang, H. Mehta, T. Duan, D. Ding, A. Bagul, C. Langlotz, K. Shpanskaya, M. P. Lungren, and A. Y. Ng, "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," *arXiv preprint arXiv:1711.05225*, 2017.

[5] T. Ozturk, M. Talo, E. A. Yildirim, U. B. Baloglu, O. Yildirim, and U. R. Acharya, "Automated detection of COVID-19 cases using deep neural networks with X-ray images," *Computers in Biology and Medicine*, vol. 121, art. 103792, 2020.

[6] I. D. Apostolopoulos and T. A. Mpesiana, "Covid-19: Automatic detection from X-ray images utilizing transfer learning with convolutional neural networks," *Physical and Engineering Sciences in Medicine*, vol. 43, no. 2, pp. 635–640, 2020.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE CVPR*, pp. 770–778, 2016.

[8] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015.

[9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *IEEE CVPR*, pp. 248–255, 2009.

[10] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, art. 60, 2019.

[11] A. Esteva, A. Robicquet, B. Ramsundar, V. Kuleshov, M. DePristo, K. Chou, C. Cui, G. Corrado, S. Thrun, and J. Dean, "A guide to deep learning in healthcare," *Nature Medicine*, vol. 25, no. 1, pp. 24–29, 2019.

[12] E. J. Topol, "High-performance medicine: The convergence of human and artificial intelligence," *Nature Medicine*, vol. 25, no. 1, pp. 44–56, 2019.

[13] C. J. Kelly, A. Karthikesalingam, M. Suleyman, G. Corrado, and D. King, "Key challenges for delivering clinical impact with artificial intelligence," *BMC Medicine*, vol. 17, no. 1, art. 195, 2019.

[14] G. Varoquaux and V. Cheplygina, "Machine learning for medical imaging: Methodological failures and recommendations for the future," *IEEE Transactions on Medical Imaging*, vol. 41, no. 9, pp. 2434–2444, 2022.

[15] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why should I trust you?": Explaining the predictions of any classifier," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.

[16] A. Tiwary and M. Kumar, "Design patterns and performance evaluation of asynchronous microservices using FastAPI," *Software: Practice and Experience*, vol. 52, no. 8, pp. 1802–1821, 2022.